



INDIA.RUNS

Build what next India runs on

Team Name

TwoSum

Problem Statement

Build an AI recruiter that ranks the best 100 candidates from 100,000 profiles relevant to the job description.

Understand recruiter intent beyond simple keyword matching.

Reject misleading and low-quality candidate profiles.

Team Leader Name

Guransh Goyal

Solution Overview

What is your proposed solution?

- Multi-stage deterministic ranking pipeline combining filtering, retrieval, semantic search, re-ranking, and explainable recommendations.

What differentiates your approach from traditional candidate matching systems?

- Recruiter-inspired weighted scoring instead of keyword matching.
- BM25 + TF-IDF + BGE + FAISS + Cross-Encoder.
- Honeypot detection with evidence-backed candidate reasoning.

JD Understanding & Candidate Evaluation

What are the key requirements extracted from the JD?

- Must-have, Like-to-have, and Do-Not-Want requirements with production AI, retrieval, ranking, and LLM expertise.

Which candidate signals are most important for determining relevance?

- JD alignment, AI experience, production evidence, behavioral signals, career trajectory, and relocation readiness.

How does your solution evaluate candidate fit beyond keyword matching?

- Semantic embeddings, production evidence, promotion trends, recruiter signals, and off-domain penalties improve relevance.

Ranking Methodology

How does your system retrieve, score, and rank candidates?

- Hard filtering → weighted scoring → adaptive shortlist → FAISS retrieval → Cross-Encoder re-ranking → Top-100 ranking.

What models, algorithms, or heuristics are used?

- BM25, TF-IDF, BGE embeddings, FAISS IndexFlatIP, Cross-Encoder, and recruiter-inspired heuristics.

How are multiple candidate signals combined into a final ranking?

- Weighted multi-factor scoring with penalties, dense retrieval refinement, and deterministic final ranking.

Explainability & Data Validation

How are ranking decisions explained?

- Top skills, career highlights, recruiter signals, concerns, and strongest supporting evidence for every recommendation.

How do you prevent hallucinations or unsupported justifications?

- No LLM during ranking; explanations use only verified candidate profile evidence.

How does your solution handle inconsistent, low-quality, or suspicious profiles?

- Detects experience mismatch, title inflation, unrealistic expertise, and honeypot candidates using hard rejection or soft penalties.

End-to-End Workflow

What is the complete workflow from JD input to ranked candidate output?

Feature engineering → Offline embeddings → Hard filtering → JD processing → Weighted scoring → Adaptive shortlist → FAISS retrieval → Cross-Encoder → Honeypot validation → Explainable Top-100 output.

Slide 7: System Architecture

- Candidate Dataset → Feature Engineering → Hard Filtering → JD Processing
- Lexical Retrieval → Weighted Scoring → Adaptive Shortlisting
- BGE Embedding Cache → FAISS Retrieval → Cross-Encoder
- Honeypot Detection → Explainability → Validator → Submission

Results & Performance

What results or insights demonstrate ranking quality?

- Semantic retrieval improves relevance while honeypot filtering and evidence-backed reasoning increase recruiter trust.

How does your solution meet the challenge's runtime and compute constraints?

- Offline embedding cache, adaptive shortlist, CPU-only inference, FAISS acceleration, ≤ 2 -minute runtime, and no network calls.

Technologies Used

What technologies, frameworks, and tools were used and why were they selected for this solution?

- **Python:** Core pipeline implementation.
- **NumPy & Pandas:** Efficient feature engineering and processing.
- **Scikit-learn & BM25:** Fast lexical retrieval and ranking.
- **Sentence Transformers (BGE):** Dense semantic embeddings.
- **FAISS:** Efficient vector similarity search.
- **Cross-Encoder MiniLM:** High-precision semantic re-ranking.
- **Streamlit, Hugging Face & GitHub:** Sandbox, model hosting, and version control.

Submission Assets

GitHub Repository: <https://github.com/GuranshGoyal/RedRob26>

Sandbox Link (Streamlit Cloud): <https://resumegs.streamlit.app/>



INDIA.RUNS

Build what next India runs on

THANK YOU